



C Programming — Complete Notes

Semester 1 · Programming for Problem Solving · Mitali Verma



Contents

1. Introduction to C
2. Structure of a C Program
3. Compilation Process
4. Data Types
5. Variables & Constants
6. Input / Output
7. Operators
8. Decision Making (if/switch)
9. Loops
10. Arrays
11. Strings
12. Functions
13. Recursion
14. Storage Classes
15. Pointers
16. Structures & Unions
17. Dynamic Memory Allocation
18. File Handling
19. Preprocessor Directives
20. Common Confusions & Exam Tips

Foundations

1 Introduction to C

What is C? C is a general-purpose, procedural (step-by-step instruction based) programming language created by **Dennis Ritchie in 1972** at Bell Labs. It's called the "mother of all languages" because C++, Java, Python's interpreter, and even parts of Windows/Linux are built using C concepts.

Why you're learning it first: C teaches you exactly how a computer thinks (memory, variables, execution order) with no hidden magic. Once C clicks, every other language (Python, Java, C++) feels easier because they all borrow C's core ideas.

Key Features

Feature	Meaning
Procedural	Code runs top to bottom, in the order you write it, using functions

Fast / Low-level access	Lets you directly control memory (via pointers)
Portable	Same C code can run on Windows, Linux, Mac with minor changes
Structured	Programs are broken into functions/blocks — easy to manage
Rich Library	Built-in functions for math, strings, files, etc. (e.g. <code>stdio.h</code>)

Used for: Operating systems (Linux kernel), embedded systems (microcontrollers, IoT devices), device drivers, game engines, and as the base for learning DSA (Data Structures & Algorithms) in interviews.

Foundations

2 Structure of a C Program

Every C program follows this skeleton:

```
// 1. Preprocessor directives – tell compiler which libraries to include
#include <stdio.h>

// 2. Global declarations (optional) – visible to whole program
int globalMarks = 90;

// 3. main() function – execution ALWAYS starts here
int main() {
    // 4. Local variable declarations
    int age = 18;

    // 5. Statements / Logic
    printf("Hello Mitali! Age: %d", age);

    // 6. Return statement – tells OS the program ended successfully
    return 0;
}
```

► Output: Hello Mitali! Age: 18

Line-by-line meaning

- `#include <stdio.h>` → gives access to **standard input output** functions like `printf` , `scanf`

- `int main()` → the compulsory entry point; `int` means it returns a whole number to the OS
- `{ }` → curly braces mark the start/end of a block of code
- `;` → every statement ends with a semicolon (forgetting this is the #1 beginner error)
- `return 0;` → 0 means "program ran successfully", non-zero usually means error

Foundations

3 How C Code Runs (Compilation Process)

C is a **compiled language** — your code is fully translated into machine code (0s and 1s) *before* it runs, unlike Python which is interpreted line-by-line.

4 Stages:

Stage	What Happens	File Produced
1. Preprocessing	Handles lines starting with <code>#</code> (like <code>#include</code> , <code>#define</code>)	<code>.i</code>
2. Compilation	Converts C code to Assembly language	<code>.s</code>
3. Assembly	Converts Assembly to machine code (object code)	<code>.o</code> / <code>.obj</code>
4. Linking	Combines your object code with library code to make final program	<code>.exe</code>

Compilers you'll use: GCC (most common, free), Turbo C (old, still used in some colleges), or online compilers like OneCompiler / Compiler Explorer.

Basics

4 Data Types

Data type tells the compiler **what kind of value** a variable will hold and **how much memory** to reserve for it.

Type	Size (typical)	Stores	Format Specifier
<code>int</code>	4 bytes	Whole numbers	<code>%d</code>
<code>float</code>	4 bytes	Decimal numbers (single precision)	<code>%f</code>
<code>double</code>	8 bytes	Decimal numbers (higher precision)	<code>%lf</code>
<code>char</code>	1 byte	A single character	<code>%c</code>
<code>void</code>	—	"Nothing" — used for functions that return no value	—

Modifiers

`short` , `long` , `signed` , `unsigned` adjust size/range. Example: `unsigned int` stores only 0 and positive numbers but a bigger positive range.

```
int age = 18;
float price = 99.5;
double pi = 3.141592653589;
char grade = 'A';
printf("%d %f %lf %c", age, price, pi, grade);
```

Trick to remember: single quotes `'A'` = char, double quotes `"A"` = string (array of chars).

Basics

5 Variables & Constants

Variable = a named memory location whose value CAN change. **Constant** = a value that CANNOT change once set.

Naming Rules

- Can contain letters, digits, underscore — must start with letter or `_`
- Cannot use C keywords (`int` , `return` , etc.) as names
- Case sensitive → `Age` and `age` are different

Declaring Constants

```
// Method 1: const keyword  
const float PI = 3.14;
```

```
// Method 2: #define (preprocessor, no memory used, no semicolon)  
#define RATE 5
```

Why constants matter: If a value like tax rate is used 50 times in code and it changes, you edit ONE line instead of 50 — reduces bugs.

Basics

6 Input & Output

`printf()` → prints output to screen. `scanf()` → reads input from user. Both come from `stdio.h`.

```
int main() {  
    int marks;  
    printf("Enter your marks: ");  
    scanf("%d", &marks); // & means "address of" marks  
    printf("You entered: %d", marks);  
    return 0;  
}
```

Why the & in scanf? `scanf` needs the actual memory *address* of the variable to store the input directly into it — not a copy. (Not needed for strings, since an array name is already an address — covered in Arrays section.)

Common Format Specifiers

Specifier	Used For
%d	int
%f	float
%lf	double

<code>%c</code>	char
<code>%s</code>	string
<code>%x</code>	hexadecimal

Basics

7 Operators

Arithmetic

`+` `-` `*` `/` `%`

`%` = modulus (remainder) → `10 % 3 = 1`

Relational

`==` `!=` `>` `<` `>=` `<=`

Result is always 1 (true) or 0 (false)

Logical

`&&` (AND), `||` (OR), `!` (NOT)

Assignment

`=` `+=` `-=` `*=` `/=` `%=`

Increment/Decrement

`++` and `--` (pre and post)

Bitwise

`&` `|` `^` `~` `<<` `>>` (work on binary bits)

Pre vs Post increment: `++a` increases first, then uses value. `a++` uses value first, then increases. Example: if `a=5`, `printf("%d", a++)` prints **5** but `a` becomes 6 right after.

Control Flow

8 Decision Making — if / else / switch

Lets the program choose a path based on a condition (true/false).

```
int marks = 75;
if (marks >= 90) {
    printf("Grade: A");
} else if (marks >= 75) {
    printf("Grade: B");
} else {
    printf("Grade: C");
}
```

► Output: **Grade: B**

switch — cleaner alternative for many fixed choices

```
int day = 3;
switch(day) {
    case 1: printf("Monday"); break;
    case 2: printf("Tuesday"); break;
    case 3: printf("Wednesday"); break;
    default: printf("Invalid day");
}
```

► Output: **Wednesday**

Never forget `break` ! Without it, execution "falls through" and runs the next cases too.

Control Flow

9 Loops

Loops repeat a block of code until a condition becomes false — used to avoid writing repetitive code.

for loop — best when you know how many times to repeat

```
for (int i = 1; i <= 5; i++) {
    printf("%d ", i);
}
```

► Output: **1 2 3 4 5**

while loop — best when repetition depends on a condition, count unknown

```
int i = 1;
while (i <= 5) {
    printf("%d ", i);
    i++;
}
```

do-while — runs the body AT LEAST ONCE, condition checked after

```
int i = 1;
do {
    printf("%d ", i);
    i++;
} while (i <= 5);
```

Statement	Effect
<code>break</code>	Exits the loop immediately
<code>continue</code>	Skips current iteration, goes to next

Data Structures

10 Arrays

An array stores **multiple values of the same type** in one variable, in continuous memory locations, accessed using an index (starting from 0).

```
int marks[5] = {90, 85, 76, 60, 99};
printf("%d", marks[0]); // prints 90 (first element)
printf("%d", marks[4]); // prints 99 (last element)

// Looping through an array
for(int i = 0; i < 5; i++){
    printf("%d ", marks[i]);
}
```


► Output: 90 85 76 60 99

2D Arrays (like a table/grid — used for matrices)

```
int matrix[2][3] = { {1,2,3}, {4,5,6} };
printf("%d", matrix[1][2]); // row 1, col 2 -> prints 6
```

Why it matters: Arrays are the base of almost every DSA topic (stacks, queues, sorting, searching) you'll study for placements.

Data Structures

1 1 Strings

In C, a string is just a **character array** ending with a special null character `'\0'` (marks the end).

```
char name[20] = "Mitali";
printf("%s", name); // %s used for whole strings
```

Common String Functions (from `string.h`)

Function	Use
<code>strlen(s)</code>	Returns length of string (excludes <code>'\0'</code>)
<code>strcpy(dest, src)</code>	Copies src string into dest
<code>strcat(s1, s2)</code>	Joins (concatenates) s2 to end of s1
<code>strcmp(s1, s2)</code>	Compares two strings; returns 0 if equal

```
#include <string.h>
char a[20] = "Hello", b[20] = "World";
strcat(a, b);
printf("%s", a); // HelloWorld
printf("%d", strlen(a)); // 10
```

Modular Code

1 2 Functions

A function is a reusable block of code that performs one task. `main()` itself is a function — every C program has at least one.

```
// Function declaration (prototype)
int add(int a, int b);

int main() {
    int result = add(5, 3); // function CALL
    printf("%d", result);
    return 0;
}

// Function definition
int add(int a, int b) {
    return a + b;
}
```

► Output: 8

Why break code into functions?

- Reusability — write once, call many times
- Easier debugging — test small pieces separately
- Readability — `main()` stays short and clear

Parameter Passing

Type	Meaning
Call by Value	A copy of the variable is sent; original is unaffected by changes inside function
Call by Reference	The address (pointer) is sent; original variable CAN be changed

Modular Code

1 3 Recursion

A function that calls **itself** to solve a smaller version of the same problem, until it reaches a "base case" that stops it.

```
int factorial(int n) {
    if (n == 0) return 1;      // base case - stops recursion
    return n * factorial(n - 1); // recursive call
}
// factorial(4) = 4*3*2*1*1 = 24
```

Why learn it: Recursion is essential for tree/graph traversal, sorting algorithms (merge sort, quick sort), and is a favorite interview/exam topic.

Golden rule: Every recursive function MUST have a base case, or it runs forever and crashes (stack overflow).

Advanced Basics

1 4 Storage Classes

Storage class decides a variable's **lifetime** (how long it exists) and **scope** (where it can be accessed).

Class	Default Value	Scope	Lifetime
<code>auto</code>	Garbage	Local (inside block)	Till block ends
<code>static</code>	0	Local	Whole program — retains value between calls
<code>extern</code>	0	Global	Whole program — declared elsewhere
<code>register</code>	Garbage	Local	Till block ends — stored in CPU register for speed

Most asked: `static` variables remember their value the next time the function is called — unlike normal local variables which reset every call.

Core Concept

1 5 Pointers

A pointer is a variable that stores the **memory address** of another variable, instead of a normal value.

```
int age = 20;
int *p = &age; // p stores the ADDRESS of age

printf("%d", age); // 20 (value)
printf("%p", &age); // e.g. 0x7ffe... (address)
printf("%d", *p); // 20 (*p means "value AT this address" - dereferencing)
```

Symbol	Meaning
&	"Address of" — gives the memory location of a variable
*	"Value at address" (dereference) OR declares a pointer variable

Why pointers are powerful (and scary at first): They let a function actually modify the original variable (call by reference), let you build dynamic data structures (linked lists, trees), and let you manage memory manually.

Beginner tip: Read `int *p` as "p is a pointer that will point to an int". Practice tracing address vs value on paper — this is where most Sem-1 students get confused, so go slow here.

Core Concept

1 6 Structures & Unions

Structure groups *different data types* together under one name — useful for representing a real-world entity (like a Student with name, roll no, marks).

```
struct Student {
    char name[20];
    int roll;
    float marks;
};

int main() {
    struct Student s1 = {"Mitali", 101, 92.5};
    printf("%s %d %.1f", s1.name, s1.roll, s1.marks);
    return 0;
}
```

► Output: Mitali 101 92.5

Union looks similar but all members **share the same memory location** — so only one member holds a valid value at a time (saves memory).

Structure	Union
Each member has own memory	All members share one memory block
Size = sum of all members	Size = size of largest member
All values usable simultaneously	Only last-assigned value is valid

Advanced

1 7 Dynamic Memory Allocation

Normally array size is fixed at compile time. DMA lets you request memory **while the program is running**, from `stdlib.h`.

Function	Use
<code>malloc(size)</code>	Allocates memory, contains garbage values
<code>calloc(n, size)</code>	Allocates memory for n elements, initialized to 0

<code>realloc(ptr, size)</code>	Resizes previously allocated memory
<code>free(ptr)</code>	Releases memory back to the system

```
int *arr = (int*) malloc(5 * sizeof(int));
if (arr == NULL) { printf("Memory not allocated"); }
arr[0] = 10;
free(arr); // always free what you malloc – prevents "memory leaks"
```

Advanced

1 8 File Handling

Lets a program store data permanently on disk (in a file) instead of losing it when the program ends.

```
FILE *fp = fopen("data.txt", "w"); // open in WRITE mode
fprintf(fp, "Mitali scored 92");
fclose(fp); // ALWAYS close after use

FILE *fp2 = fopen("data.txt", "r"); // open in READ mode
char buffer[100];
fscanf(fp2, "%[^\n]", buffer);
printf("%s", buffer);
fclose(fp2);
```

Mode	Meaning
"r"	Read (file must exist)
"w"	Write (creates new / erases existing)
"a"	Append (adds to end without erasing)

Advanced

1 9 Preprocessor Directives

Lines starting with `#` — processed BEFORE compilation, they don't end with a semicolon.

Directive	Use
<code>#include</code>	Inserts a library file's code
<code>#define</code>	Creates a macro / constant (simple text replacement)
<code>#ifdef / #ifndef</code>	Conditional compilation — includes code only if a condition is met

```
#define SQUARE(x) ((x)*(x))
printf("%d", SQUARE(5)); // 25
```

Exam Ready



Common Confusions & Quick Exam Tips

Confusion	Clarification
<code>=</code> VS <code>==</code>	<code>=</code> assigns a value; <code>==</code> compares two values
<code>char c='A'</code> VS <code>char s[]="A"</code>	First is one character; second is a 2-char array ('A' + '\0')
Array vs Pointer	An array name decays into a pointer to its first element, but an array itself is a fixed block, a pointer can be reassigned
while vs do-while	while checks condition BEFORE running; do-while runs body once, THEN checks
Global vs Local variable	Global: declared outside all functions, accessible everywhere. Local: declared inside a function, accessible only there
malloc vs calloc	malloc = uninitialized garbage memory; calloc = memory pre-set to zero

How to "own" this topic in a viva/interview: For every concept, be able to say in one line: (1) what it is, (2) a tiny code example, (3) why/where it's used. That 3-step explanation

is exactly what examiners and interviewers want — you now have all three for every topic above.

Practice plan: Don't just read — type every example above yourself in a compiler, change the numbers, and predict the output before running it. That's how this becomes permanent knowledge instead of just notes.

Made for Mitali Verma · B.Tech IT, Parul University · Semester 1, 2026